Principal Component Analysis (PCA)

1) What is PCA?

- PCA is a dimensionality reduction technique in machine learning.
- It helps reduce the number of features (columns) in your dataset while keeping most of the important information (variance).
- Benefits of PCA:-

1. Faster training :- fewer features → less computation
2. Faster inference :- model predicts quicker
3. Easier visualization :- reduce 100s of features to 2D or 3D for plotting
4. Handles curse of dimensionality - reduces model complexity & overfitting.

2) Why do we need PCA?

- Real life dataset often have hundreds or thousands of features.
- Not all features impact the target variable equally:
  1. Example :- predicting house prices
     - Important features:- area, ...

impact more

impact less more

Date
Page

area	bathroom	plot	trees nearby	price
2600	2	8500	2	550000
3000	3	9200	2	565000
3200	3	8750	2	610000
3600	4	10200	2	680000
4000	4	15000	2	725000
2600	2	7000	2	585000
2800	3	9000	2	615000
3300	4	10000	1	650000
3600	4	10500	1	710000

less

Date
Page

- less important features:- number of bathrooms
- Almost irrelevant features:- number of trees nearby

- PCA identifies the most important features, called principal components (PCs)

3. How PCA works (Conceptually)

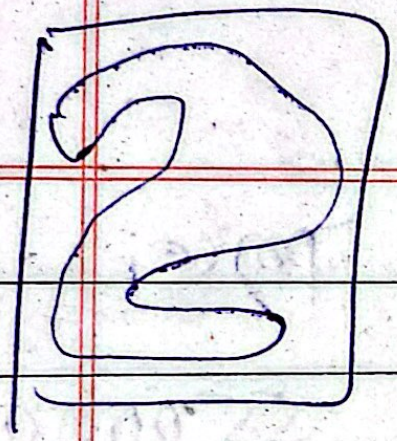
Example 1:- Digits dataset

- Handwritten digits are represented as 8×8 pixel images → 64 features
- Some pixels are always black (not important) → can be ignored
- PCA identifies which pixels (features) contribute most variance.

Example 2:- Iris flower dataset

- Scatter plot of sepal width vs sepal height.
- Draw a line along the maximum variance → principal component 1 (PC1)

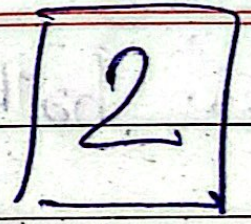
8*8 gavld



0	0	11	16	9	0	0	0
0	0	13	11	12	0	0	0
0	0	5	0	2	7	0	0
0	0	3	0	4	5	0	0
0	0	0	6	13	4	0	0
0	0	2	13	16	7	0	0
0	0	8	1	3	9	0	0
0	0	2	8	8	8	4	2

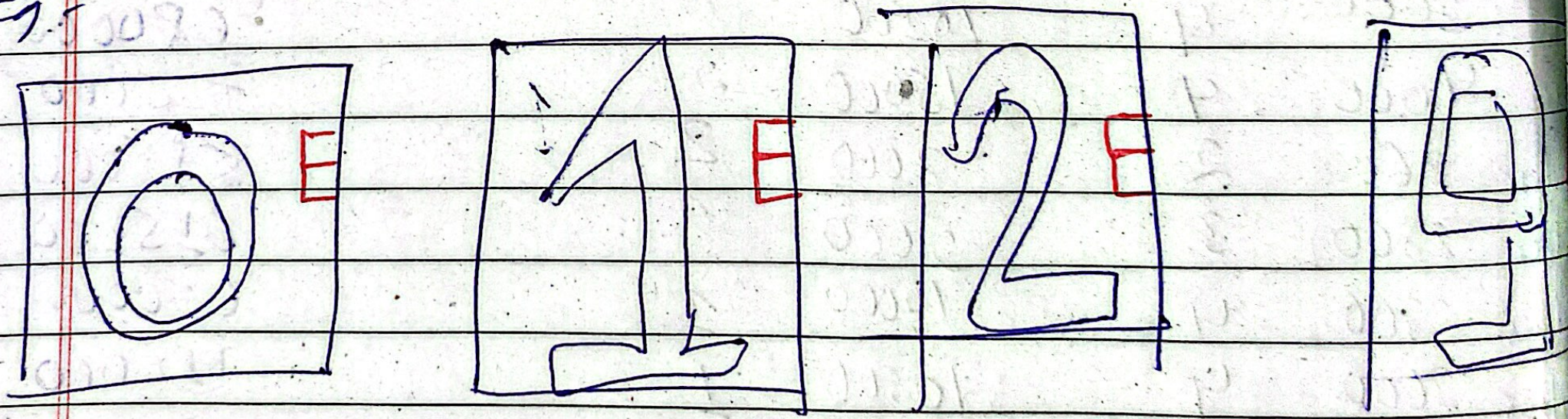
64 features

Lower pixels



Date
Page

Ex:



No, matter what the numbers these pixels are blac
are not important.

64 features ↓

SN	Pixel_0	pixel_1	pixel_2	...	pixel_63	Digit
0	0.0	0.0	5.0	...	13.0	0
1	0.0	0.0	0.0	...	12.0	2
2	0.0	0.0	0.0	...	4.0	9
3	0.0	0.0	7.0	...	15.0	1
4	0.0	0.0	0.0	...	1.0	

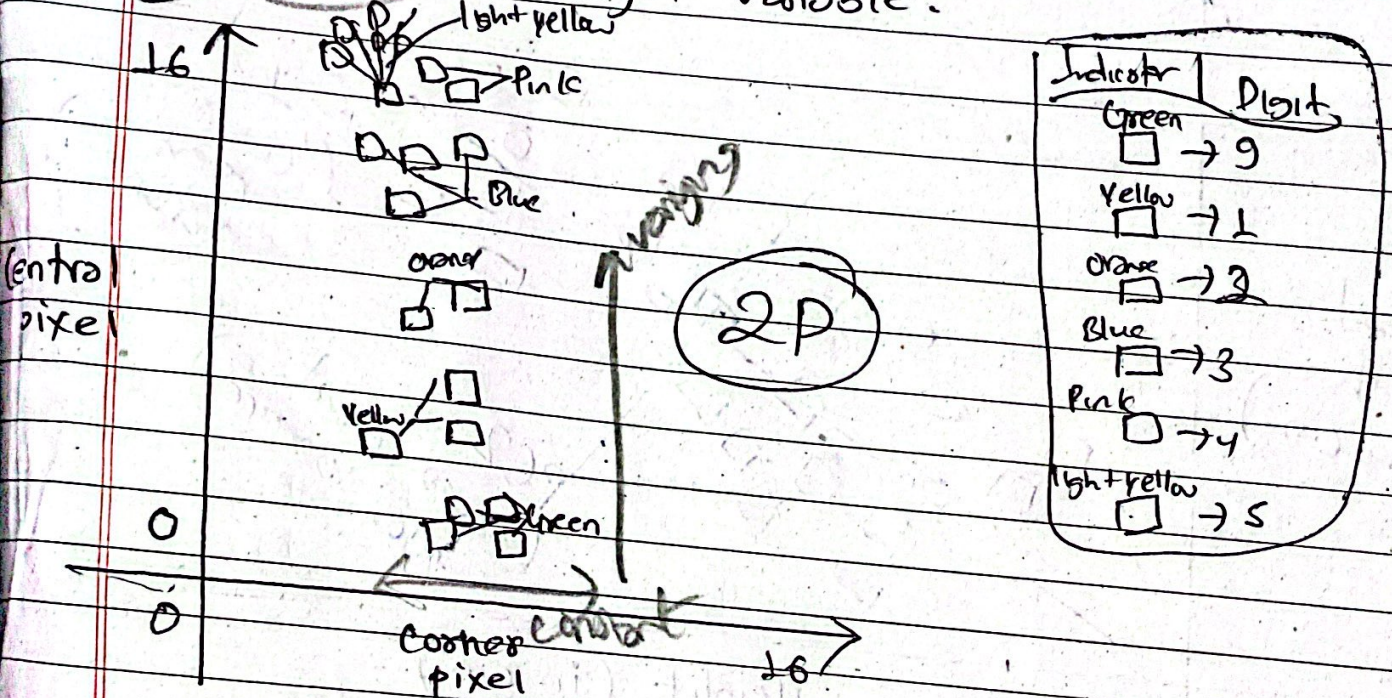
↓

PCA(n_component = 6)

↓ only 6 important features

SN	PC1	PC2	PC3	PC4	PC5	PC6	Digit
0	5.2	15.0	...	1.25	...	...	8
1	7.2	13.0		0.0	...	...	0
2	1.23	4.0		9.0	...	...	2
3	9.5	12.0		7.2	...	...	9
4	8.1	0.0		13.0	...	...	1

PCA is a process of figuring out most important features or principal components that has the most impact on the target variable.

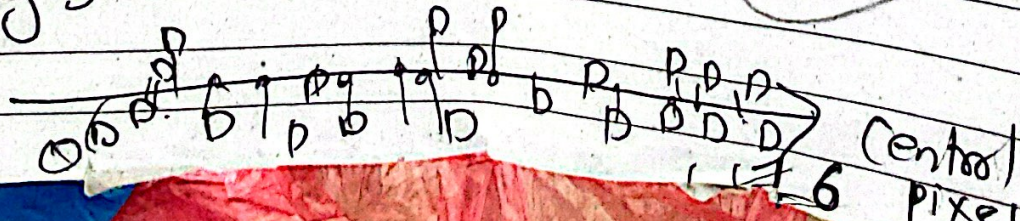


Every pixel value ranges from 0 to 16.
0 = completly black
16 = completly white

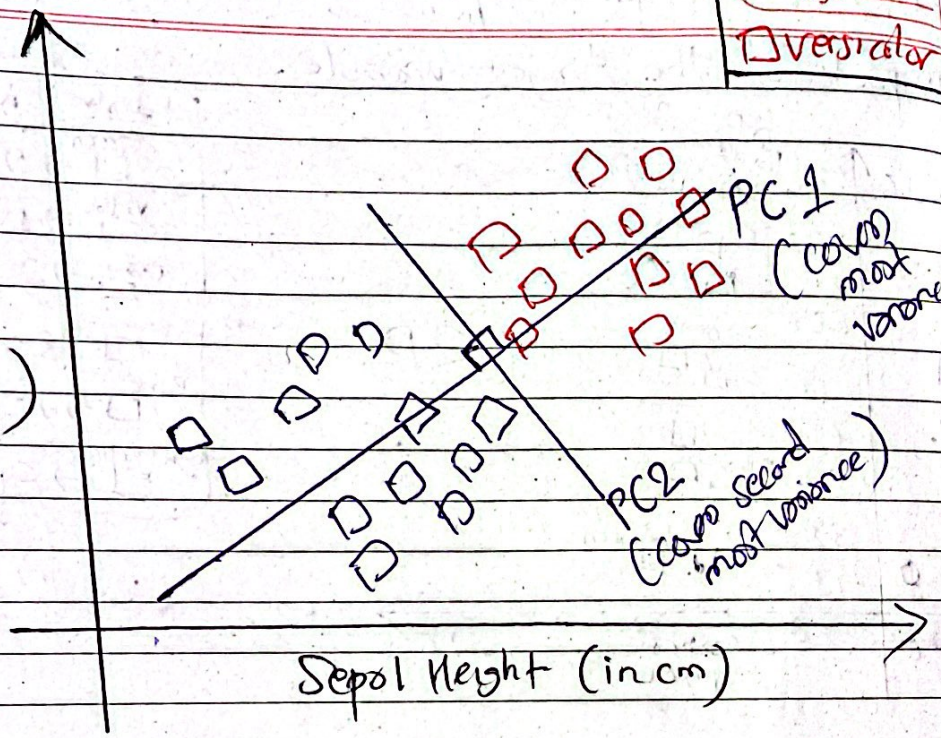
So, from the graph, we can clearly see corner pixel are almost constant (x-axis). while central pixel value are changing. (y-axis)

So, central pixel is more important & corner pixel are not that important.

So changing to 1D.



Date Page Setosa Versicolor



(learn this later in Campus X)

Math is really complex here

- Perpendicular line → principal component 2 (PC2)
- PCA orders components by variance explained.

Key points:-

PC1 → captures most variance
PC2 → second most variance
if there are 100 features, PCA can create 100 components, sorted by importance.

Important considerations before PCA

Feature scaling is necessary:-

- If features are on different scales (e.g. salary vs 1-10), PCA may give skewed result.
- Use StandardScaler or MinMaxScaler

Trade-off

- Reducing too many features → lose information → lower accuracy
- Reducing too few → may not simplify computation

6. PCA in python (Using scikit-learn)

Date
Page

Step 1:- Import libraries

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
```

Step 2:- Load dataset

digits.keys()

```
digits = load_digits()
X = digits.data      # features (64 pixels)
y = digits.target    # Target (digits 0-9)
```

Step 3:- Scale features

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Step 4:- Split data

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=4
)
```

Date
Page

Step 5:- Train baseline model

```
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
print("Accuracy without PCA", accuracy)
                                    # ~97%.
```

Step 6:- Apply PCA

```
# Retain 95% of variance
pca = PCA(n_components = 0.95)
X_pca = pca.fit_transform(X_scaled)
print("Original shape:", X_scaled.shape)
                                    # (samples, 64)
print("PCA shape:", X_pca.shape)
                                    # (samples, 29)
```

Step 7:- Explained variance

```
print(pca.explained_variance_ratio_)
# shows how much information each PC captures
```

I. principal compone

Step 8:- Train model with PCA data

```
X_train_pca, X_test_pca, y_train, y_test
    = train_test_split(
        X_pca, y, test_size=0.2, random_state=42
    )

model.fit(X_train_pca, y_train)
accuracy_pca = model.score(X_test_pca, y_test)
print("Accuracy with PCA:", accuracy_pca)
```

5. Using PCA with fixed number of components

- Example `n_components=2`

```
pca = PCA(n_components=2)
X_pca2 = pca.fit_transform(X_scaled)
print("Shape:", X_pca2.shape)  # only 2 features
print(pca.explained_variance_ratio_)
# usually lower, accuracy drops
```

- Trade-off:- fewer components → faster computation but lower accuracy

7. Key Takeaways

- PCA reduces high-dimensional data to fewer dimensions.
- Helps in:
  - Faster computation
  - Easier visualization
  - Avoiding overfitting
- Always scale features before PCA.
- Choose number of components based on variance or trial-and-error.
- Accuracy usually remains high if enough variance is retained.

Variance In PCA

- In PCA, variance refers how spread out the data is along each feature.
- features (columns) with high variance contain more information because they differ a lot between data points.
- features with low variance or almost constant across data points → they don't add much information.

Example:-

- Predicting house prices
  - Area ranges from 500 to 5000 sq ft → high variance → important feature.
  - Number of trees nearby is 2 or 3 for almost all houses → low variance → not very important

Why PCA uses variance:-

- PCA identifies directions (principal components) in which the data varies the most.
- By keeping components with high variance, PCA retains most of the important information & ignores unimportant features.

In short! -

- Variance in PCA = amount of information / spread in the data.
- PCA keeps the components / features with maximum variance because they capture the essence of the dataset.

Outliers vs high variance

- Outliers are datapoints that are far away from the rest of the data.
- Outliers can artificially increase variance, making PCA think a feature is important when most of the data doesn't vary much.
- This can distort the principal components & affect results.

Example: - • Dataset of house prices mostly between $50k - $200k.
- One house costs $1 million → huge outlier.
- PCA sees that feature (price) has very high variance because of the outlier.
- But in reality, most data points don't vary much → PCA might overemphasize that outlier.

Soln/Best Practice -

- Scale features (Standard or MinMax scaler) → reduces impact of large numbers.
- Detect & remove outliers before PCA.

→ Variance is good to measure information, but extreme variance (outliers) can misleads PCA. Always check data distribution before PCA.